

CIF Microarchitecture Specification

Block: Transaction Tag Allocator

CIF Design Team

June 18, 2026

Contents

1	Overview	2
2	Design Parameters	2
3	Internal Blocks	2
3.1	Occupancy Counter Array	2
3.2	Seqnum Counter Array	2
3.3	Stall Logic	3
3.4	Allocate Logic	3
3.5	Free Logic	3
4	Interface Signals	3
4.1	Inputs	3
4.2	Outputs	4
5	Functional Behaviour	4
5.1	Allocation Flow	4
5.2	Stall Flow	4
5.3	Free Flow	4
6	Timing Notes	5
7	Edge Cases	5

1 Overview

The Transaction Tag Allocator is a shared combinational/registered block instantiated once in the CIF and used by both the read and write paths across all `N_CLIENTS = 32` AXI clients.

Its responsibilities are:

- Allocate a unique `txn_tag[7:0]` for every validated transaction entering the pipeline.
- Enforce the per-AXID outstanding transaction limit of `MAX_OUTSTANDING = 4`.
- Stall upstream (Request Validator / Error Handler) on a per-AXID basis when the outstanding limit is reached.
- Free an occupied slot when a transaction completes — via `BRESP` on the write path, or `rob_last` on the read path.

Tag encoding (agreed with MC Core team):

```
txn_tag[7:0]:
[7:2] = AXID[5:0]    -- 6-bit AXI ID, up to 64 unique IDs
[1:0] = seqnum[1:0]  -- per-AXID slot index, wraps at 4
```

2 Design Parameters

Parameter	Value	Description
AXID_W	6	AXI ID width; 64 unique IDs
MAX_OUTSTANDING	4	Max outstanding txns per AXID
SEQNUM_W	2	Seqnum width ($\log_2$ MAX_OUTSTANDING)
TXN_TAG_W	8	Tag width = AXID_W + SEQNUM_W
N_AXIDS	64	Total AXID space (2^6)
OCC_CTR_W	2	Occupancy counter width per AXID

Table 1: Transaction Tag Allocator parameters

3 Internal Blocks

3.1 Occupancy Counter Array

- **Structure:** 64 independent 2-bit saturating counters, one per AXID.
- **Increment:** On every successful allocation (`alloc_en` asserted), the counter for the requesting AXID is incremented.
- **Decrement:** On assertion of `WR_FREE_VALID` or `RD_FREE_VALID`, the counter for the AXID extracted from the incoming tag (`tag[7:2]`) is decremented.
- **Read port:** Continuously exposes `occupancy[AXID]` to Stall Logic.

3.2 Seqnum Counter Array

- **Structure:** 64 independent 2-bit wrap counters, one per AXID. Represents the *next* seqnum to be allocated.
- **Increment:** On every successful allocation (`alloc_en` asserted), the counter for the requesting AXID is incremented modulo 4.

- **Read port:** Continuously exposes `seqnum[1:0]` for the requesting AXID to Allocate Logic.
- **No decrement:** Seqnum counters are allocation-only; they are never decremented. The occupancy counter separately tracks the number of in-flight slots.

3.3 Stall Logic

- **Type:** Purely combinational.
- **Function:** Asserts `STALL` for a given AXID when `occupancy[AXID] == MAX_OUTSTANDING(4)`.
- **Granularity:** Per-AXID. Only the AXID currently presented by the upstream block is evaluated; other AXIDs are unaffected.
- **Output:** `STALL` fed back to Request Validator and Error Handler.

3.4 Allocate Logic

- **Gate condition:** Fires only when `REQ_VALID` is asserted and `STALL` is deasserted (`alloc_en = REQ_VALID & ~STALL`).
- **Tag formation:**

<code>TXN_TAG[7:0] = { AXID[5:0], seqnum[1:0] }</code>
--

- **Side-effects on alloc:** `inc_occ` to Occupancy Counter Array; `inc_seq` to Seqnum Counter Array.
- **Output:** `TXN_TAG[7:0]` and `TAG_VALID` forwarded to Burst Splitter.
- **Error path:** Error Handler presents AXID with `REQ_VALID` asserted; allocation proceeds identically. The resulting tag is used to form a NOP packet (`pkt_type = Err_write / Err_read`) in the normal pipeline. No special casing in Allocate Logic.

3.5 Free Logic

- **Two free ports:**
 - Write path: `WR_FREE_VALID`, `WR_FREE_TAG[7:0]` — sourced from Response Tracker on `BRESP` emission.
 - Read path: `RD_FREE_VALID`, `RD_FREE_TAG[7:0]` — sourced from Read Transaction Table on `rob_last` assertion.
- **AXID extraction:** `AXID = tag[7:2]` in both cases.
- **Action:** Asserts `dec_occ` to Occupancy Counter Array for the extracted AXID.
- **Simultaneous frees:** Both ports may assert in the same cycle for different AXIDs — handled independently. Simultaneous frees on the same AXID (one write + one read) are not possible by construction (a tag is either a read or a write transaction, never both).

4 Interface Signals

4.1 Inputs

Signal	Width	Source	Description
REQ_VALID	1	Req Validator / Error Handler	Allocation request valid
AXID[5:0]	6	Req Validator / Error Handler	Requesting AXI ID
WR_FREE_VALID	1	Response Tracker	Write path free strobe
WR_FREE_TAG[7:0]	8	Response Tracker	Tag being freed (write)
RD_FREE_VALID	1	Read Txn Table	Read path free strobe
RD_FREE_TAG[7:0]	8	Read Txn Table	Tag being freed (read)

Table 2: Tag Allocator input signals

4.2 Outputs

Signal	Width	Destination	Description
TXN_TAG[7:0]	8	Burst Splitter	Allocated transaction tag
TAG_VALID	1	Burst Splitter	Tag output valid
STALL	1	Req Validator / Error Handler	Per-AXID stall signal

Table 3: Tag Allocator output signals

5 Functional Behaviour

5.1 Allocation Flow

1. Upstream presents AXID[5:0] and asserts REQ_VALID.
2. Stall Logic reads occupancy[AXID] from the Occupancy Counter Array combinatorially.
3. If occupancy[AXID] < 4: STALL is deasserted, alloc_en is asserted.
4. Allocate Logic reads seqnum[1:0] from the Seqnum Counter Array for the requesting AXID.
5. TXN_TAG = {AXID, seqnum} is driven; TAG_VALID is asserted.
6. Occupancy Counter Array increments for AXID.
7. Seqnum Counter Array increments (mod 4) for AXID.

5.2 Stall Flow

1. If occupancy[AXID] == 4: STALL is asserted to upstream.
2. Upstream holds REQ_VALID and AXID stable.
3. Stall is released as soon as Free Logic decrements occupancy for that AXID (i.e., on the next WR_FREE_VALID or RD_FREE_VALID for a tag belonging to that AXID).

5.3 Free Flow

1. Response Tracker asserts WR_FREE_VALID with WR_FREE_TAG when a BRESP is emitted (write path).
2. Read Txn Table asserts RD_FREE_VALID with RD_FREE_TAG when rob_last fires (read path).

3. Free Logic extracts $AXID = tag[7:2]$ and asserts `dec_occ` to the Occupancy Counter Array.
4. This may immediately release a stall on that AXID if Stall Logic observes the updated occupancy.

6 Timing Notes

- Stall Logic is purely combinational: `STALL` is available in the same cycle that AXID is presented.
- Allocate Logic is registered: `TXN_TAG` and `TAG_VALID` are valid on the cycle *following* `alloc_en`.
- Counter updates (`inc/dec`) are registered and take effect on the following clock edge. A free in cycle N releases the stall in cycle $N+1$.

7 Edge Cases

- **Simultaneous alloc and free on same AXID:** If `alloc_en` and `dec_occ` fire in the same cycle for the same AXID, occupancy is held constant (net zero change). This is a valid steady-state operating condition.
- **Error path:** Error Handler allocates a tag identically to valid transactions. The resulting tag is embedded in a NOP packet (`pkt_type = Err_write / Err_read`). The slot is freed normally on MC Core error completion.
- **Underflow protection:** The Occupancy Counter Array must not decrement below zero. This is guaranteed by construction — a free is only issued when a tag is in-flight — but a design-time assertion should be included in the verification environment.